

ROS2 Tutorials - ROS2 Humble For Beginners

Piotr Sarna

March 2026

Contents

1	Intro: Install and Setup ROS2 Humble - Tutorial 1	2
2	Start Your First ROS2 Node - Tutorial 2	3
3	Create and Set Up a ROS2 Workspace - Tutorial 3	4
4	Create a ROS2 Python Package - Tutorial 4	5
5	Create a ROS2 Node with Python and OOP - Tutorial 5	6
6	What is a ROS2 Topic? - Tutorial 6	7
7	Write a ROS2 Publisher with Python - Tutorial 7	8
8	Write a ROS2 Subscriber with Python - Tutorial 8	9
9	Create a Closed Loop System with a Publisher and a Subscriber - Tutorial 9	10
10	What is a ROS2 Service? - Tutorial 10	11
11	Write a ROS2 Service Client with Python - Tutorial 11	12

1 Intro: Install and Setup ROS2 Humble - Tutorial 1

To install packages in ROS2, use the following command syntax:

```
sudo apt install ros-humble-package_name
```

Environment Setup

To configure the environment, you need to edit the terminal configuration file. Open the file using the GNOME text editor (`gedit`):

```
gedit ~/.bashrc
```

- `gedit` – GNOME text editor.
- `~/.bashrc` – Terminal configuration file.

Add the following line at the end of the `~/.bashrc` file:

```
source /opt/ros/humble/setup.bash
```

- `source` – Applies settings in this file to the currently open terminal.
- `/opt/ros/humble/setup.bash` – ROS script which sets up commands, paths, etc.

Summary

The combination of these steps ensures that the `ros2` command works always in any opened terminal.

2 Start Your First ROS2 Node - Tutorial 2

Key Concepts

ROS2 Node – ROS2 program, which interacts with ROS2 communications and tools.

ROS2 Package – ROS2 Nodes organized in packages.

Writer Listener Demo

To start the demo, use the `ros2 run` command followed by the package name and node name:

```
ros2 run demo_nodes_cpp talker
```

- `demo_nodes_cpp` – Package name.
- `talker` – Node name.

To start the listener node, open a new terminal and run:

```
ros2 run demo_nodes_cpp listener
```

Turtlesim Demo

To run the turtlesim demo, start the main node and the teleoperation node:

```
ros2 run turtlesim turtlesim_node  
ros2 run turtlesim turtle_teleop_key
```

ROS2 Tools

ROS2 tool, shows running Nodes:

```
rqt_graph
```

3 Create and Set Up a ROS2 Workspace - Tutorial 3

Colcon – build tool for ROS2, builds code and Nodes.

Update the list of software available in external libraries:

```
sudo apt update
```

Install the colcon tool:

```
sudo apt install python3-colcon-common-extensions
```

Colcon Autocompletion

Add the following line at the end of the `~/.bashrc` file to enable autocompletion for colcon arguments:

```
source /usr/share/colcon_argcomplete/hook/colcon-argcomplete.bash
```

Workspace Setup

To set up a new workspace, create the workspace folder and the source folder:

```
mkdir ros2_ws  
mkdir ros2_ws/src
```

Inside the `ros2_ws` folder, run the command to build the ROS2 workspace in the current folder:

```
colcon build
```

Sourcing the Workspace

Add the following line at the end of the `~/.bashrc` file to allow using custom ROS2 Nodes in any terminal created in this workspace:

```
source ~/workspace_name/install/setup.bash
```

4 Create a ROS2 Python Package - Tutorial 4

In `src` folder:

Create a ROS2 Package:

```
ros2 pkg create package-name --build-type
```

Specify if it's a C++/Python package using the build type:

- `ament_cmake` – for C++
- `ament_python` – for Python

Add rcl (ROS Client Library) API:

```
--dependencies
```

- `rclcpp` – for C++
- `rclpy` – for Python

Note

In every package, there is a `package.xml` file containing the name, dependencies, build-type, etc., and a similar `setup.py` file.

In workspace folder:

Build the package, needed before starting nodes:

```
colcon build
```

5 Create a ROS2 Node with Python and OOP - Tutorial 5

In `package_name/package_name`:

Create Node:

```
touch node_name.py
```

Make Node executable:

```
chmod +x node_name.py
```

Python Node Implementation:

```
1 #!/usr/bin/env python3
2 # Tell interpreter to use Python3
3
4 import rclpy # Import Python library for ROS2
5 from rclpy.node import Node # Created Nodes inherit from built-in Node class
6
7 class MyNode(Node): # MyNode inherits from Node
8     def __init__(self):
9         super().__init__("first_node") # names Node
10        self.counter_ = 0
11        self.create_timer(1.0, self.timer_callback)
12
13        def timer_callback(self):
14            self.get_logger().info("Hello " + str(self.counter_))
15            self.counter_ += 1
16
17 def main(args=None):
18     rclpy.init(args=args) # initializes ROS2 communications
19     node = MyNode()
20     rclpy.spin(node) # node will continue to run until killed
21     rclpy.shutdown() # terminates ROS2 communications
22
23 if __name__ == '__main__': # import protection
24     main()
```

In `setup.py`:

Add to entry points:

```
1 entry_points={
2     'console_scripts': [
3         'test_node = package_name.file_name:main'
4     ],
5 }
```

- `test_node` – executable name.
- `main` – function name.

Run updated version of Node:

```
colcon build --symlink-install
source ~/.bashrc
ros2 run package_name node_name
```

- `--symlink-install` – run updated version without build.

List running Nodes:

```
ros2 node list
```

Show info about a Node:

```
ros2 node info /node_name
```

6 What is a ROS2 Topic? - Tutorial 6

Communication Flow

`/talker` (Publisher Node) → `/chatter` (Topic) → `/listener` (Subscriber Node)

List running Topics:

```
ros2 topic list
```

Show info about a Topic:

```
ros2 topic info /topic_name
```

Type: `std_msgs/msg/String`

Publisher count: 1

Subscription count: 1

Show details of a message interface:

```
ros2 interface show std_msgs/msg/String
```

`string data` – sent message contains data of 'string' type.

Show data received by Topic:

```
ros2 topic echo /chatter
```

Note

There can be multiple Nodes publishing to the same Topic and multiple Nodes subscribing to the same topic.

7 Write a ROS2 Publisher with Python - Tutorial 7

Import message type:

```
1 from geometry_msgs.msg import Twist
```

In `package.xml` add imported module:

```
<depend>geometry_msgs</depend>
```

In constructor:

```
1 self.cmd_vel_pub_ = self.create_publisher(Twist, "/turtle1/cmd_vel", 10)
```

- `self.cmd_vel_pub_` – Create publisher.
- `Twist` – Message type.
- `"/turtle1/cmd_vel"` – Topic name.
- `10` – queue size.

`geometry_msgs/msg/Twist` consists of:

- `linear` (position)
- `angular` (rotation)

Publishing function:

```
1 def send_velocity_command(self):  
2     msg = Twist()  
3     msg.linear.x = 2.0  
4     msg.angular.z = 1.0  
5     self.cmd_vel_pub_.publish(msg) # send msg via publisher
```


8 Write a ROS2 Subscriber with Python - Tutorial 8

Import message type:

```
1 from turtlesim.msg import Pose
```

In `package.xml` add imported module:

```
<depend>turtlesim</depend>
```

In constructor:

```
1 self.pose_publisher_ = self.create_subscription(Pose, "/turtle1/pose", self.pose_callback, 10)
```

- `self.create_subscription` – Create subscriber.
- `Pose` – Message type.
- `"/turtle1/pose"` – Topic name.
- `self.pose_callback` – Callback name.
- `10` – Queue size.

Note

Callback will be called everytime when you receive a message on that Topic.

Callback function receiving messages:

```
1 def pose_callback(self, msg: Pose):  
2     self.get_logger().info("(" + str(msg.x) + ", " + str(msg.y) + ")")
```

9 Create a Closed Loop System with a Publisher and a Subscriber - Tutorial 9

Combine previous two tutorials – Create a Node that both subscribes and publishes appropriate command when received a message.

10 What is a ROS2 Service? - Tutorial 10

Services are interactions that act like a Client/Server. Client sends a request to the Server and Server responds to the Client. Topics don't give answers.

Services cannot be seen in `rqt_graph`. List running services:

```
ros2 service list
```

See Service type:

```
ros2 service type /service_name
```

Example: `example_interfaces/srv/AddTwoInts`

See data inside Service message:

```
ros2 interface show example_interfaces/srv/AddTwoInts
```

```
int64 a - request
int64 b - request
---
int64 sum - response
```

Make a request:

```
ros2 service call /service_name service/type "{a: 2, b: 5}"
```

- Response on client from server: `(...) sum = 7`
- Info about request on server: `(...) a: 2 b: 5`

Server/Client Architecture

There can only be one server, but many clients connecting to the server.

Requests are typically used for:

- Computation
- Change of settings

Topics vs Services

Topics are used to send straight data, without expecting an answer.

11 Write a ROS2 Service Client with Python - Tutorial 11

Import module:

```
1 from functools import partial
```

In `package.xml` add imported module:

```
<depend>functools</depend>
```

Create a Service-sending function:

```
1 def call_set_pen_service(self, argument):
2     client = self.create_client(ServiceType, "/service_name")
3
4     # Wait for service to be available
5     while not client.wait_for_service(1.0):
6         self.get_logger().warn("Waiting for service")
7
8     request = ServiceType.Request()
9     request.argument = argument
10
11    # Request without waiting for response
12    future = client.call_async(request)
13    future.add_done_callback(partial(self.callback))
```

Using partial

`partial` is used when, for example, a callback needs more arguments than ROS can give at the moment. `partial` allows adding missing arguments in the meantime.

Callback function:

```
1 def callback(self, future):
2     try:
3         response = future.result()
4     except Exception as e:
5         self.get_logger().error("Response error")
```

Topics vs Services Usage

You can publish on a Topic very frequently. A Service is made to be called only when needed. Otherwise, it can really slow down the application.

Get frequency of a Topic:

```
ros2 hz /topic_name
```